

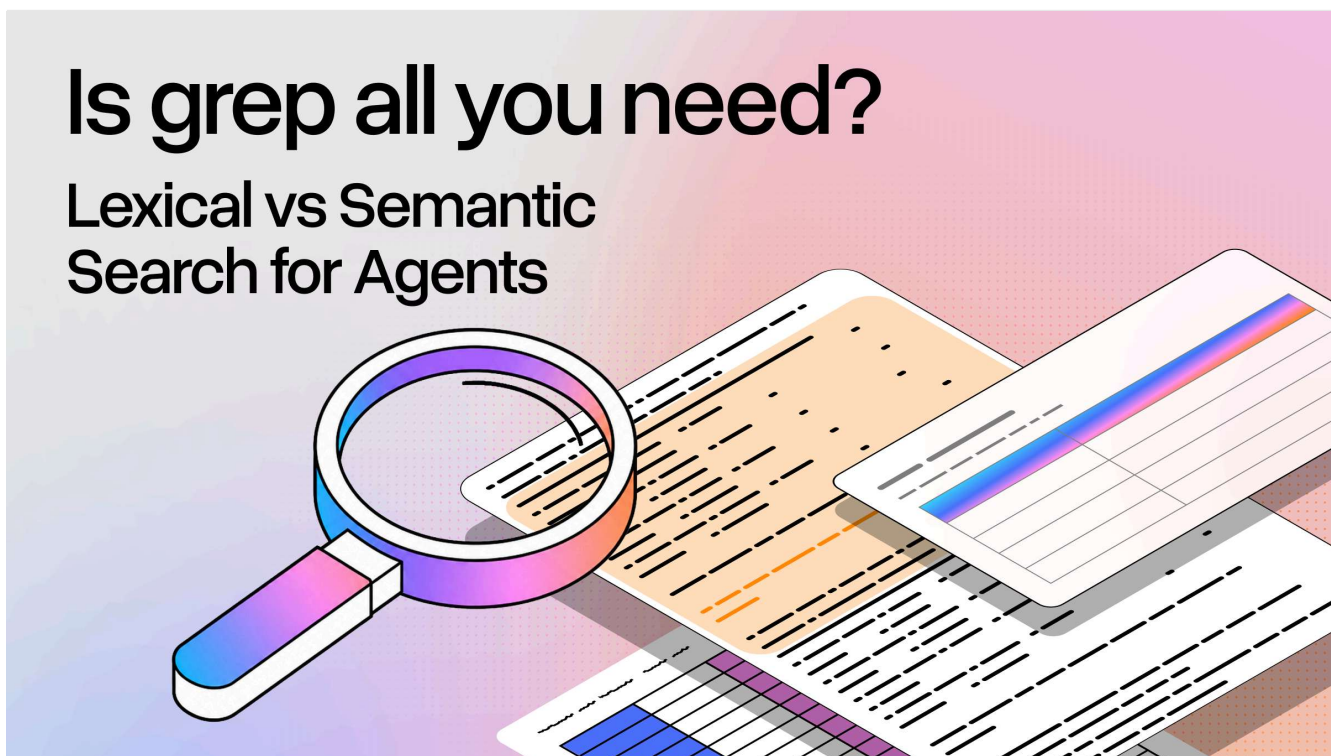
MAY 26, 2026 ■ [SEARCH] [RAG]

Is grep all you need? Lexical VS Sematic Search for Agents

BY



CLELIA ASTRA BERELLI



BUILDING FOR SCALE: SEMANTIC SEARCH AND RAG ▼



1

In a [recent paper](#), Sen et al. argued that `grep` might be the best interface for a world where search is heavily reshaped by agent harnesses. The idea that filesystem tools will soon overthrow semantic search, and RAG in general, has been circulating for a

while, but the debate is mostly structured around text-based documents (markdown files, source code), and rarely accounts for what agents actually encounter day-to-day in most enterprise settings: unstructured documents (PDFs, Office files, images).

In this post, we'll look at how and when `grep` and, more generally, lexical search can be faster and more accurate than RAG, and when it isn't.

Grep and what comes with it

Lexical search with `grep` rests on two assumptions:

- A text corpus is available, typically as a filesystem of text-based files.
- The corpus is small (hundreds to thousands of documents), so an agent can pick which files to search without drowning in a low signal-to-noise ratio.

In most setups, `grep` is exposed as a bash tool, assuming the agent has access to a shell.

`grep` is an excellent tool for precise substring and regex matching. It has no semantic representation of the query, which is fine as the agent manipulates semantics itself by issuing different search patterns across calls. This makes it great for retrieving highly specific information: a known token, a function name, an error string. Context is king, and the agent rarely uses `grep` in isolation: it searches for a passage, then expands context by reading the surrounding file.

`grep` has also been around for more than 50 years, so examples of its use are everywhere, which lets LLMs apply it effectively and generalize across patterns and optimizations.

Still, lexical search has two big limitations:

- **You can't `grep` a PDF, an image with text, or an Office document.** The corpus lexical search unlocks is exclusively plain text. Despite the growing adoption of markdown

and other text formats, most enterprise knowledge remains locked behind unstructured files.

- **Scalability falls apart at corpus size.** The original `grep` takes more than 4 seconds to scan 1,000,000 files for a small pattern. Even with sub-second alternatives like `ripgrep`, the noise from random, out-of-scope matches quickly fills the agent's context window and pushes relevant information out.

For both limitations, there are enterprise-grade approaches that improve the scalability of agentic search while preserving accuracy and latency.

Unlocking unstructured documents

Most CLI agent harnesses today ship with rudimentary tools for reading PDFs, plus multimodal capabilities for "seeing" images.

PDF reading in coding agents, though, is generally inaccurate and lossy: layout-unaware extractors clump tables together, ignore images, and shred columns. Vision is more expensive at inference time, and since models are primarily trained on text, vision-based reading is slower and more prone to errors and hallucinations.

A tooling layer that balances accuracy and latency is required to unlock unstructured documents and expose their text content to downstream tools like `grep`. LlamaIndex offers a set of agent-native tools for this:

- The LlamaParse MCP is a plug-and-play MCP server that lets agents call the LlamaParse platform for parsing, splitting, and classifying files, with support for 130+ formats and strong accuracy on tables, charts, images, complex layouts, and handwritten text — driven by agentic OCR.
- LiteParse is a fast, fully local tool for parsing unstructured files. It extracts text spatially, preserving layout, and uses OCR (Tesseract or a custom plug-and-play HTTP server) to faithfully represent the content. Response times are typically a few seconds. LiteParse is ideal for quick local workloads where the agent needs a fast

overview of a document, and it's the best companion for `grep` — it can write to stdout or to files that can then be searched.

LlamaIndex



- Both LiteParse and LlamaParse have agent skills that can be installed with the `vercel` CLI or pulled directly from GitHub. The skills give your agent the context it needs to use LiteParse, the LlamaParse SDK, and the MCP effectively from day one.

Once you've unlocked unstructured documents, you can combine that knowledge with the right kind of agentic search (lexical or semantic), which is the subject of the next section.

Building for scale: semantic search and RAG

Lexical search breaks down well before you reach enterprise scale. When the corpus grows from thousands to millions of documents (internal wikis, contracts, support tickets, research reports, design specs) `grep`-style search degrades on three axes at once:

- **Latency.** A linear scan over a million files is too slow for an interactive agent loop, even with `ripgrep`. Every additional retry or refined query multiplies the cost.
- **Recall.** Lexical search only finds what was literally typed. Ask for "revenue recognition" and you'll miss documents that say "ASC 606", "booking rules", or "when we record sales". The agent has to know the vocabulary used in the corpus, which defeats the point of search.
- **Signal-to-noise.** At a million documents, even a specific token will return thousands of incidental matches. The relevant ones get buried, and the agent's context window fills up with junk before it can reason.

Semantic search (and the broader RAG pattern built on top of it) sidesteps all three. Documents are parsed once (with a layout-aware tool like LlamaParse for unstructured formats), chunked, embedded into a vector space, and indexed. At query time, the agent's natural-language question is embedded too, and an approximate-nearest-

neighbor index returns the top-k semantically related chunks in tens of milliseconds, regardless of whether the corpus has ten thousand or ten million documents.

This is where the scalability story really lives:

- **Sub-linear retrieval.** ANN indexes (HNSW, IVF, ScaNN) keep query time roughly constant as the corpus grows. A million-document index returns results in the same wall-clock budget as a ten-thousand-document one.
- **Vocabulary-agnostic recall.** Embeddings capture meaning, so "revenue recognition" matches "ASC 606" without the agent having to enumerate synonyms. This dramatically reduces the number of retries the agent has to make.
- **Bounded context cost.** Top-k retrieval gives the agent a small, ranked set of chunks instead of an unbounded list of `grep` hits. The context window stays clean, and the agent can spend its tokens reasoning instead of filtering.
- **Hybrid is even better.** Production RAG systems combine semantic search with lexical (BM25) and metadata filters, getting the precision of exact-match search with the recall of embeddings.

Semantic search requires an indexing pipeline, an embedding model, and a vector store, and it's less precise than `grep` when you know the exact string you're looking for. But once the corpus is large, heterogeneous, or full of unstructured documents, those costs pay for themselves on the very first query.

Conclusion: is `grep` all you need?

`grep` is not going away, and it shouldn't. For small, plain-text corpora (a codebase, a docs folder, a handful of markdown notes) lexical search is fast, predictable, and gives agents exactly the precision they need.

But "is `grep` all you need?" is the wrong question for the world most enterprise agents actually live in. The corpus is millions of documents, most of them are unstructured (PDFs, slides, spreadsheets, scans), and the queries are framed in natural language

rather than known tokens. There, lexical search alone hits a wall on every axis that matters: it can't read the formats, it doesn't scale, and it can't bridge vocabulary gaps.

The pragmatic answer is layered. Parse unstructured documents into faithful text with a layout-aware tool like LlamaParse or LiteParse. Index that text for semantic search so the agent can retrieve by meaning at scale. Keep `grep` in the toolbox for the cases where exact-match search on a known corpus is genuinely the right call, and let the agent choose between them.

`grep` is a great tool. It's just not the only one your agent needs.

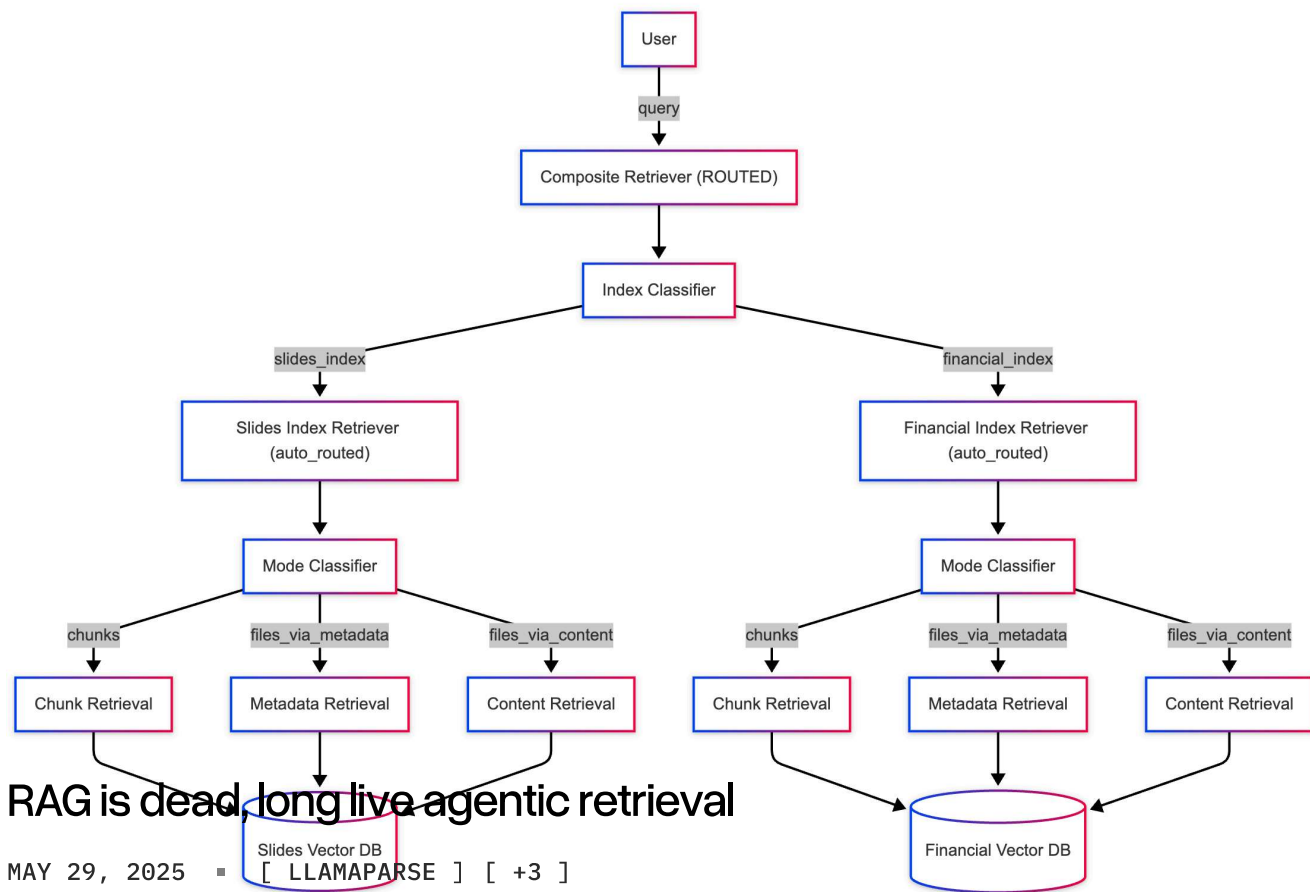
RELATED ARTICLES

An abstract image featuring several dark, three-dimensional geometric shapes, including rectangles and triangles, scattered across a black background. The shapes are illuminated from the side, creating soft highlights and shadows that give them a sense of depth and volume. Overlaid on this image is the title 'Did Files Kill Vector Search?' in a large, white, sans-serif font.

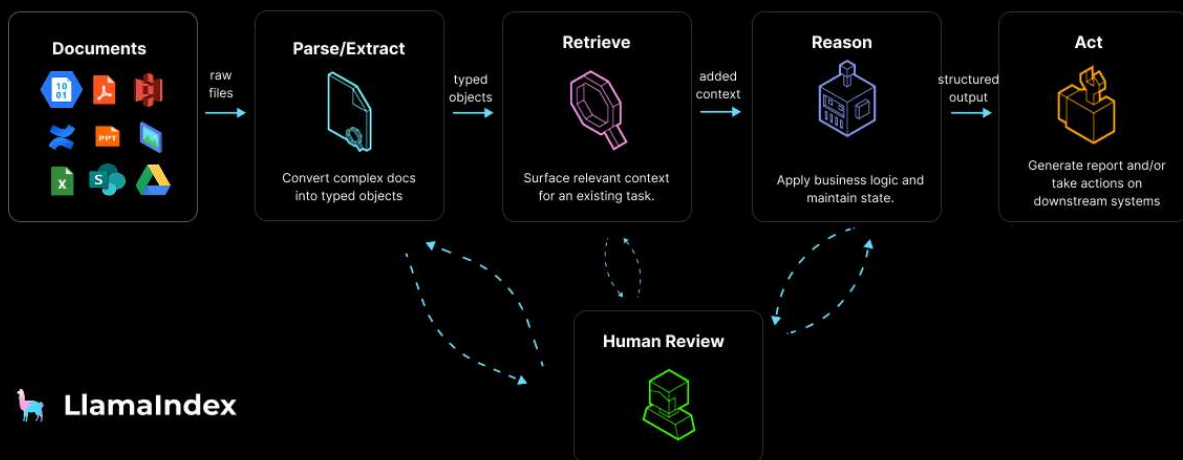
Did Files Kill Vector Search?

Did Filesystem Tools Kill Vector Search?

JAN 13, 2026 ■ [RAG] [+1]



Agentic Document Workflows



Beyond chatbots: adopting Agentic Document Workflows for enterprises

APR 23, 2025 ■ [AGENTIC DOCUMENT WORKFLOWS] [+2]

Start building your first document agent today

[SIGN UP FOR FREE](#)

[BOOK A DEMO](#)



Build document agents that understand, reason, and act

CONTACT SALES

SIGN UP

Explore AI Summary



SOLUTIONS

Engineering & R&D

Administrative Operations

Financial Analysts

Developers

Insurance

Finance

Manufacturing

Healthcare & Pharma

Finance Due Diligence

Invoice Processing

Technical Document Search

Customer Support

PRODUCTS

LlamaParse

Parse

Extract

Index

LlamaIndex

Workflows

RESOURCES

Customer Stories

Glossary

Applications

Insights

COMPANY

Pricing

Blog

About us

Careers

Brand

Trust center

WEEKLY NEWSLETTER

Get a weekly roundup of the latest news and insights on the world of LLMs and word on the newest features of the LlamaIndex libraries.

For example, "jane@company.com"

SUBSCRIBE



LlamaIndex